

Futuristic HUD-Style UI for JARVIS

¹ The existing JARVIS GUI uses a dark, *cyberpunk* aesthetic with neon accents and console-style data readouts. This matches the “HUD-style” look of modern sci-fi interfaces. As noted by the Arwes framework (“a web framework to build UIs based on futuristic science fiction designs, animations, and sound effects” ¹), influences like *Tron: Legacy* and other cyber-futuristic films are common in such designs ². The screenshots show screens for “ACCESS REQUIRED” (locked) and “ACCESS ACCEPTED” states with bright cyan text on black, animated scanlines, and metric panels (CPU, RAM, etc.), much like a film-grade spacecraft console. This suggests using techno or monospace fonts (e.g. Orbitron, Exo, JetBrains Mono as in the Python GUI), neon colors (cyan/magenta), and animated overlays (scanlines, glitch effects) to reinforce the theme.

Sci-Fi UI Patterns and Toolkits

³ Cyberpunk and futuristic UIs typically use *vibrant neon colors and glow effects* to feel high-tech. For example, one design guide notes that cyberpunk interfaces “often feature vibrant and intense neon colors, particularly in shades of blue, pink, and purple” to evoke futuristic cityscapes ³. Typography is similarly sci-fi – think angular, geometric or “glitchy” digital fonts – to enhance the advanced vibe ⁴. This aligns with the ARWES framework, which explicitly cites retro-futuristic media (e.g. *Halo*, *TRON: Legacy*) as inspiration ² ¹.

- **Sci-Fi CSS/JS Frameworks:** Use a modern library tailored for this style. For instance, [Arwes](#) provides React/Vanilla components and animations for futuristic HUDs ¹. Arwes is still in alpha, but its *concepts* (glowing backgrounds, animated lines, sci-fi sound cues) are directly applicable. Another option is [Cybercore CSS](#), a *pure-CSS* “cyberpunk” framework inspired by *Blade Runner* and *Cyberpunk 2077* ⁵. Cybercore offers out-of-the-box neon outlines, glitch text, scanline overlays, and themed components (buttons, cards, terminals) that match our UI needs ⁶. For example, a **SYSTEM STATUS** panel can be created with `<div class="cyber-card">` and neon badges as shown in its docs ⁷. These toolkits handle the heavy lifting of styling so we can focus on layout.

- **Animation Libraries:** Animated transitions are key for an “intro” effect. CSS `@keyframes` or JS libraries (like [Anime.js](#) or [GSAP](#)) can animate startup sequences (fade-ins, scanning lines, text typewriter effects). For voice and system feedback, Arwes even supports sound effects with its animations. Overall, combine CSS3 animations (blinks, scans) with JavaScript timing to simulate the system boot and login flow.

UI Components and Interaction Flow

- **Intro/Boot Sequence:** On launch, play a short animation (“JARVIS initializing...”) before showing the login screen. For example, display animated text lines (“Powering up... Calibrating sensors...”) with a scanning overlay on the logo. Use CSS animations or a canvas background animation to show

moving grids or particles (the “stars and lines” fade seen in one screenshot). After ~3 seconds, transition to the authentication screen.

- **Authentication Screen:** Mirror *SYSTEM LOCKED* mode from the screenshots. Show a prominent “ACCESS REQUIRED” message, a security level (e.g. “7/7”), and a face-recognition panel. The face panel might simply say “No Face Detected” or show a camera icon. Use neon-bordered panels (`div` with glowing outline) and a blinking cursor or pulsing alert on the “Unauthorized” text. Once the user activates (e.g. by saying “JARVIS” or scanning face), animate in the next panel.
- **Main HUD/Status Panels:** After login, display the *System State* and *Mission Core* screens like in the second screenshot. Key components:
 - A **System Status** card: shows CPU, RAM, Disk usage bars or numeric readouts. This can be a vertical panel with labelled values (cyan text on black).
 - A **Battery** and **Disk** indicator with neon bars (e.g. a horizontal bar that’s 96% filled in magenta for Disk, green for battery level).
 - A **Live Feed** panel: an HTML `<video>` element streaming the webcam (framed with neon border).
 - A **Voice Chat Status** indicator (“Ready for voice, camera, and file-aware chat” with a microphone icon).
 - A **Clock/Date** display in a stylized font.
- Possibly a **Mission Control** overlay with rotating 3D elements (could use CSS transform or a simple animated icon). Arrange these in a grid or flex layout. For example, the left side could hold system stats (like a vertical panel), the right side the camera feed, and top-center the chat prompt area.
- **Chat/Command Panel:** At bottom or overlay, include a text input and log view. Similar to Discord or Slack but styled as a dark transparent panel. Use `textarea` styled with neon outline for input, and display AI/human messages above. Support attaching files/images: add a paperclip button that opens an `<input type="file">`. When a file is attached, it’s sent to the AI (multimodal prompt). This follows modern AI chat UIs where “Attach document/image” is built-in ⁸.
- **Notifications/Toasts:** For events (e.g. “Access Granted”, “File Uploaded”), show temporary toast messages in one corner. Style them with colored borders (green for success, red for alert) as in the `ToastManager` class of the Python GUI. These can fade in/out with CSS transitions.
- **Interaction Flow:**
 - **On load:** play intro animation, then show lock screen.
 - **Wake word:** The interface listens for “JARVIS” (Web Speech API) to unlock or activate.
 - **Login:** On “ACCESS ACCEPTED”, reveal the main HUD.
 - **Chatting:** User types or speaks queries; attached images/files are sent to the AI.
 - **Live feedback:** UI updates system stats live (simulated or from real sensors), and camera feed can capture images on command.

Prototype Implementation (HTML/CSS/JS)

Below is an outline of the HTML/CSS/JS structure. We use Cybercore for base styling (via CDN) and custom CSS for layout/animations.

```
<head>
  <!-- Include a cyberpunk CSS theme (Cybercore) 9 -->
  <link rel="stylesheet" href="https://unpkg.com/cybercore-css@latest/dist/
cybercore.min.css">
  <style>
    body { margin:0; background:#000; color:#00f0ff; font-family: 'Exo 2', sans-
serif; }
    .hud-panel { border: 1px solid cyan; padding:10px; margin:5px; }
    .glow-text { text-shadow: 0 0 10px cyan, 0 0 20px cyan; }
    @keyframes scanline {
      0% { background-position: 0 0; }
      100% { background-position: 0 100%; }
    }
    .scan-overlay {
      animation: scanline 2s linear infinite;
      background: linear-gradient(rgba(255,255,255,0.05), transparent);
      mix-blend-mode: screen;
    }
  </style>
</head>
<body>
  <!-- Intro Animation Overlay -->
  <div id="intro" class="hud-panel glow-text" style="position:absolute;top:
40%;width:100%;text-align:center;">
    JARVIS SYSTEM BOOTING...<br><span class="scan-
overlay">||||||||||||||||||</span>
  </div>

  <!-- Main HUD, hidden until intro finishes -->
  <div id="hud" style="display:none;">
    <header class="hud-panel glow-text">JARVIS • {<span id="time"></span>}</
header>
    <div style="display:flex;">
      <!-- Left: System Status -->
      <div id="status" class="hud-panel">
        <h4 class="cyber-h5">SYSTEM STATE</h4>
        <p>CPU: 10% | RAM: 82%</p>
        <p>Disk: <span style="color:#ff00ff">96%</span></p>
        <p>Battery: <span style="color:#05ffa1">25%</span></p>
      </div>
      <!-- Center: Camera Feed -->
```

```

    <div class="hud-panel">
      <h4 class="cyber-h5">CAMERA FEED</h4>
      <video id="video" width="320" height="240" autoplay style="border:1px
solid #0ff;"></video>
    </div>
    <!-- Right: Chat Panel -->
    <div class="hud-panel" style="flex:1;">
      <h4 class="cyber-h5">CHAT</h4>
      <div id="chatWindow" style="height:150px; overflow-y:auto;
background:#111;"></div>
        <textarea id="chatInput" placeholder="Type or say something..."
          style="width:100%;background:#000;color:#0ff;"></textarea>
        <button class="cyber-btn">Send</button>
        <button class="cyber-btn cyber-btn--outline">Attach</button>
      </div>
    </div>
  </div>

<script>
  // Show current time
  function updateTime() {
    document.getElementById('time').textContent = new Date().toLocaleTimeString();
  }
  setInterval(updateTime, 1000);
  updateTime();

  // Simulate boot sequence then reveal HUD
  setTimeout(() => {
    document.getElementById('intro').style.display = 'none';
    document.getElementById('hud').style.display = 'block';
    // Show access granted toast
    alert("ACCESS GRANTED");
  }, 3000);

  // Access webcam feed using Web API (requires HTTPS) 10
  navigator.mediaDevices.getUserMedia({video: true})
    .then(stream => {
      document.getElementById('video').srcObject = stream;
    })
    .catch(err => console.error("Camera error:", err));
</script>

```

This mockup uses:

- **Cybercore CSS** for cyberpunk colors/effects ⁹. It defines `.cyber-h5`, `.cyber-btn`, etc., out of the box (see Cybercore docs for classes ⁷).

- **Custom CSS** for neon glow and scanline animation. The `.scan-overlay` div animates a faint white line vertically, mimicking a CRT scan.
- **JavaScript** to manage the intro timing and live updates. For example, `setTimeout` hides the intro div and shows the main HUD after 3 seconds.
- **Camera Feed:** uses `navigator.mediaDevices.getUserMedia({video:true})` to stream the webcam into `<video>` ¹⁰. The MDN docs confirm `getUserMedia()` prompts for permission and returns a video stream ¹⁰.
- **Voice Activation:** (Not shown above) would use the Web Speech API. For instance, one could use `window.SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition` and then `new SpeechRecognition()` to listen continuously ¹¹. This matches MDN's demo code pattern for speech recognition ¹¹, which we'd configure to trigger when the user says "JARVIS".

Throughout, UI elements (cards, buttons, inputs) leverage the Cybercore classes to get the neon outlines and scanline backgrounds. Toast/pop-up notifications (like "ACCESS GRANTED") can use CSS transitions or a small JS library (or simple `alert()` as placeholder). Cybercore even provides neon badges and terminals for chat windows if needed ⁷ ¹².

Implementation Notes and Integration

⁸ To support multimodal prompts, the chat interface should allow **attaching files/images**. The OpenWebUI documentation highlights "Attach documents, images, and code for the AI to analyze" in modern chat UIs ⁸. In practice, the "Attach" button can open a file dialog; selected files are sent to the Jarvis backend (OpenRouter) along with text. The backend should handle these attachments (e.g. OCR images or parse text) before querying the AI model.

Integration steps:

- **Backend API:** Use the existing Python `jarvis_gui` code or a Node server to accept chat messages and file uploads. The frontend can POST to an API endpoint, which calls the OpenRouter multi-model chat backend.
- **Camera/Audio:** We already stream video via `getUserMedia` ¹⁰. For audio, the Web Speech API (`SpeechRecognition`) can capture voice commands continuously ¹¹. For example, `recognition.continuous = true; recognition.onresult = ...` can feed text into the chat input whenever a phrase is recognized.
- **Deployment:** This UI can be a web app (hosted with HTTPS for media permissions). Optionally, package it as an Electron desktop app for cross-platform use. Ensure to secure access (the "SECURITY LEVEL 7/7" panel implies strict auth).

By combining these elements—neon-glow styling (via Cybercore), sci-fi animations (via CSS/JS), and real-time media APIs—we can create a full-featured Jarvis assistant interface that matches the provided designs. The prototype code above can be expanded: using actual data feeds for CPU/RAM (via `os` metrics in Python or a Node addon), fleshing out the face-recognition/login flow, and integrating the full AI chat backend.

Sources: Established sci-fi UI frameworks and design guidelines ¹ ³ ⁵ ⁸ ¹⁰ ¹¹ inform the layout, look, and tech stack of this enhanced JARVIS GUI.

1 GitHub - arwes/arwes: Futuristic Sci-Fi UI Web Framework. · GitHub

<https://github.com/arwes/arwes>

2 Arwes

<https://arwes.dev/docs>

3 4 6.5 of The Most Popular UI Design Trends and Styles Explained

<https://designerup.co/blog/here-are-6-5-of-the-most-popular-ui-design-trends-and-how-to-design-them/>

5 6 7 9 12 Introducing CYBERCORE CSS: A Cyberpunk Design Framework for Futuristic UIs - DEV Community

<https://dev.to/sebyx07/introducing-cybercore-css-a-cyberpunk-design-framework-for-futuristic-u-is-2e6c>

8 ★ Features / Open WebUI

<https://docs.openwebui.com/features/>

10 MediaDevices: getUserMedia() method - Web APIs | MDN

<https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>

11 Using the Web Speech API - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API/Using_the_Web_Speech_API